torch.fft.ifftn#

https://pytorch.org/docs/stable/generated/torch.fft.ifftn.html

Description:

Computes the N dimensional inverse discrete Fourier transform of input. Supports torch.half and torch.chalf on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimensions. input (Tensor) – the input tensor s (Tuple[int], optional) – Signal size in the transformed dimensions. If given, each dimension dim[i] will either be zero-padded or trimmed to the length s[i] before computing the IFFT. If a length -1 is specified, no padding is done in that dimension. Default: s = [input.size(d) for d in dim]

Function Signature

```
torch.fft.ifftn(input, s=None, dim=None, norm=None, *,
out=None) → Tensor#

Parameters

Keyword Arguments
```

Parameters

Keyword

out (Tensor, optional) – the output tensor.

Code Examples

```
>>> x = torch.rand(10, 10, dtype=torch.complex64)
>>> ifftn = torch.fft.ifftn(x)
```

```
>>> two_iffts = torch.fft.ifft(torch.fft.ifft(x, dim=0), dim=1)
>>> torch.testing.assert_close(ifftn, two_iffts,
check_stride=False)
```

Notes & Warnings

NOTE

Note Supports torch.half and torch.chalf on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimensions.

Detailed Documentation

Documentation

Computes the N dimensional inverse discrete Fourier transform of input.

Supports torch.half and torch.chalf on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimensions.

input (Tensor) – the input tensor

s (Tuple[int], optional) – Signal size in the transformed dimensions. If given, each dimension dim[i] will either be zero-padded or trimmed to the length s[i] before computing the IFFT. If a length -1 is specified, no padding is done in that dimension. Default: s = [input.size(d) for d in dim]

dim (Tuple[int], optional) – Dimensions to be transformed. Default: all dimensions, or the last len(s) dimensions if s is given.

norm (str, optional) –

Normalization mode. For the backward transform (ifftn()), these correspond to:

"forward" - no normalization

"backward" - normalize by 1/n

"ortho" - normalize by 1/sqrt(n) (making the IFFT orthonormal)

Where n = prod(s) is the logical IFFT size. Calling the forward transform (fftn()) with the same normalization mode will apply an overall normalization of 1/n between the two transforms. This is required to make ifftn() the exact inverse.

Default is "backward" (normalize by 1/n).

out (Tensor, optional) – the output tensor.

The discrete Fourier transform is separable, so ifftn() here is equivalent to two one-dimensional ifft() calls: